

Group 9 Assignment Report

An Evolutionary Algorithm for the Travelling Salesman Problem

Christos Zampounis Mehmet Cihan Yalçın Péter Dávid Hajdu
Pierre Kasparian Prajval Parthiban

September 30, 2026

1 Introduction

The goal of this assignment is to design, implement and experimentally evaluate an evolutionary algorithm for a combinatorial optimisation problem. We chose the Travelling Salesman Problem, whose discrete, permutation-based search space and relevance to routing and logistics make it a natural and challenging testbed for evolutionary computation. This report presents our problem formulation, the evolutionary approach and operators we developed, and an experimental analysis of its solution quality and convergence behaviour.

2 The Travelling Salesman Problem

2.1 Mathematical Formulation

The Travelling Salesman Problem (TSP) is a classical NP-hard combinatorial optimisation problem: given a set of N cities and the distances between them, the task is to find the shortest closed tour that visits every city exactly once. Following the formulation of Potvin [1], we model the problem on a complete weighted graph $G = (V, E)$, where $V = \{1, \dots, N\}$ is the set of cities and d_{ij} is the distance between cities i and j . The decision variables are $x_{ij} \in \{0, 1\}$, where $x_{ij} = 1$ if and only if the tour travels directly from city i to city j . The objective is to minimise the total travelled distance,

$$\text{Minimise} \quad \sum_{i=1}^N \sum_{j=1}^N d_{ij} x_{ij}, \quad (1)$$

subject to the degree constraints

$$\sum_{j=1}^N x_{ij} = 1, \quad i = 1, \dots, N, \quad \sum_{i=1}^N x_{ij} = 1, \quad j = 1, \dots, N, \quad (2)$$

which require every city to have exactly one incoming and one outgoing edge, and the subtour-elimination constraints

$$\sum_{i,j \in S} x_{ij} \leq |S| - 1, \quad \forall S \subset V, \quad 2 \leq |S| \leq N - 1, \quad (3)$$

which forbid partial cycles over any proper subset S of the cities.

Because an evolutionary algorithm operates on orderings rather than on binary edge variables, it is convenient to state the equivalent tour-as-permutation formulation. A candidate solution is a permutation $\pi = (\pi_1, \dots, \pi_N)$ of the cities, and its cost is the length of the cycle that visits them in that order,

$$C(\pi) = \sum_{k=1}^{N-1} d_{\pi_k, \pi_{k+1}} + d_{\pi_N, \pi_1}. \quad (4)$$

The evolutionary algorithm therefore seeks a permutation minimising Equation (4).

2.2 Related Evolutionary Approaches

Liu [2] builds a genetic algorithm around a crossover operator called *Edge Swapping* (ES), which keeps the edges shared by the two parent tours and swaps in the cycles where they disagree; seeding the population with greedy tours refined by 2-opt, it reaches best-known solutions on large benchmarks. Liao et al. [3] propose GPSO, a genetic-based particle swarm optimisation in which each particle is a real-valued vector decoded into a tour by ranking its components, initialised geometrically by sorting cities by their angle around the geographical centre, which produces a non-crossing route. For the path representation, where a chromosome is the visiting order and classical one-point or uniform crossovers cannot be used as they duplicate cities, Hussain et al. [4] compare partially mapped (PMX), order (OX) and cycle crossover, where a segment of one parent is copied into the child and the remaining cities are filled in their order of appearance in the other parent; no operator dominates on TSPLIB.

2.3 Motivation of the Developed Approach

The evolutionary approach we develop follows the genetic algorithm presented in Section 9.2 of Mitchell [6], adapted to the permutation representation of the TSP; its pseudo-code is given in Section 3. Within this template, the design choices left open concern the initialization of the population and the crossover operator, which we align with the literature reviewed in Section 2.2. We test four initializations: random, greedy nearest-neighbour tours refined by 2-opt [2], geometric non-crossing tours [3], and their combination. For recombination we adopt the order crossover (OX) [4]. The remaining operators are left unchanged, so that the effect of the two adapted components can be isolated in Section 4.

3 Algorithm

Each individual is represented as a permutation of all cities, where the order of the cities defines the tour and the final city is connected back to the first. The initial population consists of p tours generated by the selected initialization method. The cost of an individual is given by Equation (4). Since the TSP is a minimisation problem, this cost is converted into a maximisation-based fitness function defined as

$$F(\pi) = \frac{1}{C(\pi)}. \quad (5)$$

Consequently, shorter tours receive higher fitness values and are favoured during selection. In the implementation, tour costs are used directly where possible, since minimizing $C(\pi)$ is equivalent to maximizing $F(\pi)$.

In each generation, approximately a fraction $(1-r)$ of the population is retained for the next generation: each survivor is chosen independently from the current population by tournament selection of size k , and is copied to the new population before mutation. The remaining fraction is filled with offspring generated from selected parent pairs.

Parent selection for crossover is performed using tournament selection. For each parent, k individuals are sampled randomly from the current population, and the individual with the lowest tour cost is selected. The process is repeated to obtain the second parent. Tournament selection provides selection pressure towards shorter tours while still allowing different individuals to participate in reproduction.

Order Crossover (OX) is used to generate offspring because it preserves valid permutations. A segment from one parent is copied to the offspring, while the remaining cities are inserted

according to their order in the second parent, skipping already used cities. Two offspring are generated for each parent pair by swapping the parent roles.

After crossover, mutation is applied to a fraction m of the new population. The individuals to be mutated are selected uniformly at random. Depending on the experiment, either swap mutation or inversion mutation is used. Swap mutation exchanges two randomly selected positions in a tour, whereas inversion mutation reverses the order of the cities within a randomly selected segment. Both mutation operators preserve a valid permutation.

Once mutation has been applied, the new population replaces the previous one and all tour costs are evaluated again. The best tour found so far is stored throughout the run. This means that even if a previously good individual is later modified by mutation or disappears from the population, the best solution encountered during the search is not lost.

The process is repeated for G generations. After termination, the best tour encountered during the complete run is returned.

Algorithm 1 Genetic Algorithm for the Travelling Salesman Problem

Require: cities D , population size p , replacement rate r , mutation rate m , number of generations G , tournament size k **Ensure:** best tour found $P \leftarrow$ generate p initial toursevaluate the cost of each individual in P $best \leftarrow$ individual in P with minimum cost**for** $g = 1$ to G **do** create new generation $P_s \leftarrow \emptyset$ **Selection:** $q \leftarrow \text{round}(rp/2)$ $s \leftarrow p - 2q$ **for** $i = 1$ to s **do** select an individual from P using tournament selection of size k add it to P_s **end for** **Crossover:** **for** $i = 1$ to q **do** select h_1 from P using tournament selection of size k select h_2 from P using tournament selection of size k $o_1 \leftarrow \text{ORDERCROSSOVER}(h_1, h_2)$ $o_2 \leftarrow \text{ORDERCROSSOVER}(h_2, h_1)$ add o_1 and o_2 to P_s **end for** **Mutation:** uniformly select mp individuals from P_s **for** each selected individual π **do** apply the selected mutation operator to π **end for** **Update:** $P \leftarrow P_s$ **Evaluation:** evaluate the cost of each individual in P **if** the best individual in P has lower cost than $best$ **then** update $best$ **end if** **end for** **return** $best$

4 Experimental part

4.1 Hardware and Software

All experiments were run on two MacBook Pro laptops with Apple Silicon (M-series) processors. The algorithm is implemented in Python 3.12 using only the standard library; `matplotlib` is used solely to produce the figures. Every trial runs as a single process. Random numbers are

generated from explicitly assigned seeds, making the experiments reproducible with the same code and configuration.

4.2 Experimental Protocol

Each configuration is repeated for 30 runs. The study proceeds in three stages. First, random, nearest-neighbour tours improved by 2-opt, angle-based tours, and their combination are compared on burma14, berlin52, kroA100, and kroA200. Second, population size $p \in \{50, 100, 120, 150\}$, number of generations $G \in \{100, 300, 500, 800\}$, mutation rate $m \in \{0.05, 0.1, 0.2, 0.4\}$, and replacement rate $r \in \{0.3, 0.5, 0.7, 0.9\}$ are varied one at a time on the same four instances and for all four initialization methods. The remaining parameters are held at $p = 100$, $G = 500$, $m = 0.1$, and $r = 0.7$, and inversion mutation is used. Third, the problem-size experiment uses random initialization and these default parameter values on all six instances.

4.3 Datasets

The evaluation uses six symmetric TSPLIB instances [5] of increasing size: burma14 (14 cities), berlin52 (52), kroA100 (100), kroA200 (200), pcb442 (442) and pr1002 (1002). They are committed to the repository and loaded with their original TSPLIB edge weights (GEO for burma14, EUC_2D for the others). Performance is measured as the optimality gap $(C_{\text{best}} - C_{\text{opt}})/C_{\text{opt}} \cdot 100$, using the published optima 3323, 7542, 21282, 29368, 50778 and 259045. The main study uses the four first datasets while the scaling experiment covers all six instances.

5 Results and Analysis

First we wanted to compare the effect of the initial population so we tested four different initialization methods on the same four TSPLIB instances with the same parameter settings.

Instance	Random	NN + 2-opt	Angle-based	Smart
burma14	0.50 ± 1.34	0.00 ± 0.00	0.20 ± 0.20	0.00 ± 0.00
berlin52	9.20 ± 3.09	0.00 ± 0.00	10.71 ± 3.73	0.00 ± 0.00
kroA100	44.52 ± 7.20	1.55 ± 0.50	24.46 ± 4.76	1.88 ± 0.14
kroA200	162.69 ± 8.65	2.08 ± 0.28	70.99 ± 5.23	1.96 ± 0.34

Table 1: Mean final optimality gap (%) $\pm$ sample standard deviation over 30 runs for each initialization method and instance. Lower gaps indicate better solutions.

The nearest-neighbour and smart methods already reached their known optimum in their initial populations on burma14 and berlin52. On kroA100 and kroA200, their mean initial gaps were about 1.91% and 2.25%, respectively, and the GA reduced these gaps by less than 0.4 percentage points. In comparison, the random and angle-based initializations began further from the optimum, but the GA reduced their gaps by a substantial amount during the search. This shows that initialization method had a strong effect on the final solution quality.

In order to examine how the algorithm performs across different size datasets, we ran it on the six TSPLIB benchmarks using random initialization and the same parameter settings. Table 2 reports the mean optimality gap and its sample standard deviation, along with the number of runs that reached the known optimum. A lower gap indicates a tour closer to the optimum.

Instance	Cities	Mean gap (%) $\pm$ SD	Runs at optimum
burma14	14	0.51 ± 1.16	18/30
berlin52	52	11.25 ± 3.27	0/30
kroA100	100	46.63 ± 6.23	0/30
kroA200	200	160.85 ± 12.38	0/30
pcb442	442	442.42 ± 12.28	0/30
pr1002	1002	1086.32 ± 16.16	0/30

Table 2: Optimality gap across 30 runs per TSPLIB instance using random initialization. SD is the sample standard deviation of the recorded gaps; lower gaps are better.

The mean gap increased across the instances, from 0.51% to 1086.32%. The algorithm reached the known optimum in 18 of 30 burma14 runs, but failed to reach the optimum on all the other instances. This suggests that the GA struggled to find near-optimal tours on larger benchmarks.

6 Conclusions

References

- [1] J.-Y. Potvin, “Genetic algorithms for the traveling salesman problem,” *Annals of Operations Research*, vol. 63, no. 3, pp. 339–370, 1996. <https://doi.org/10.1007/BF02125403>.
- [2] S. Liu, “A Powerful Genetic Algorithm for Traveling Salesman Problem,” arXiv:1402.4699, 2014. <https://arxiv.org/abs/1402.4699>.
- [3] Y.-F. Liao, D.-H. Yau, and C.-L. Chen, “Evolutionary algorithm to traveling salesman problems,” *Computers & Mathematics with Applications*, vol. 64, no. 5, pp. 788–797, 2012. <https://doi.org/10.1016/j.camwa.2011.12.018>.
- [4] A. Hussain, Y. S. Muhammad, M. N. Sajid, I. Hussain, A. M. Shoukry, and S. Gani, “Genetic algorithm for traveling salesman problem with modified cycle crossover operator,” *Computational Intelligence and Neuroscience*, vol. 2017, Article ID 7430125, 7 pages, 2017. <https://doi.org/10.1155/2017/7430125>.
- [5] G. Reinelt, “TSPLIB—A Traveling Salesman Problem Library,” *ORSA Journal on Computing*, vol. 3, no. 4, pp. 376–384, 1991. <https://doi.org/10.1287/ijoc.3.4.376>.
- [6] T. M. Mitchell, *Machine Learning*. New York, NY, USA: McGraw-Hill, 1997, ch. 9, sec. 9.2. <https://www.cs.cmu.edu/~tom/files/MachineLearningTomMitchell.pdf>.